

Supervised Learning Classification

Supervised learning is a **subcategory of machine learning and artificial intelligence**. It is defined by its use of labeled datasets to train algorithms that to classify data or predict outcomes accurately.

In supervised learning, algorithms learn from labeled data. After understanding the data, the algorithm determines which label should be given to new data by associating patterns to the unlabeled new data.

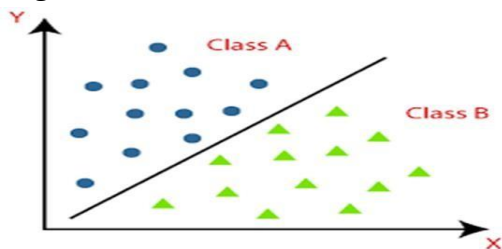
Supervised learning can be divided into two categories: classification and regression.

Supervised Learning Classification

A classification problem is when the output variable is a category, such as “Red” or “blue”, “disease” or “no disease”.

Classification predicts the category the data belongs to.

Some examples of classification include spam detection, churn prediction, sentiment analysis, dog breed detection and so on.



Types of Classification tasks in ML

There are perhaps four main types of classification tasks that you may encounter; they are:

- Binary Classification
- Multi-Class Classification
- Multi-Label Classification
- Imbalanced Classification

Binary Classification

- Binary classification refers to those classification tasks that have two class labels.
- Examples include:
 - Email spam detection (spam or not).
 - Churn prediction (churn or not).
 - Conversion prediction (buy or not).
- Typically, binary classification tasks involve one class that is the normal state and another class that is the abnormal state.
- For example “*not spam*” is the normal state and “*spam*” is the abnormal state. Another example is “*cancer not detected*” is the normal state of a task that involves a medical test and “*cancer detected*” is the abnormal state.

- The class for the normal state is assigned the class label 0 and the class with the abnormal state is assigned the class label 1.

Multi-Class Classification

- Multi-class classification refers to those classification tasks that have more than two class labels.
- Examples include:
 - Face classification.
 - Plant species classification.
 - Optical character recognition.
- Unlike binary classification, multi-class classification does not have the notion of normal and abnormal outcomes. Instead, examples are classified as belonging to one among a range of known classes.
- The number of class labels may be very large on some problems. For example, a model may predict a photo as belonging to one among thousands or tens of thousands of faces in a face recognition system.

Multi-Label Classification

- Multi-label classification refers to those classification tasks that have two or more class labels, where one or more class labels may be predicted for each example.
- Consider the example of photo classification, where a given photo may have multiple objects in the scene and a model may predict the presence of multiple known objects in the photo, such as “*bicycle*,” “*apple*,” “*person*,” etc.
- This is unlike binary classification and multi-class classification, where a single class label is predicted for each example.

Imbalanced Classification

- Imbalanced classification refers to classification tasks where the number of examples in each class is unequally distributed.
- Typically, imbalanced classification tasks are binary classification tasks where the majority of examples in the training dataset belong to the normal class and a minority of examples belong to the abnormal class.
- Examples include:
 - Fraud detection.
 - Outlier detection.
 - Medical diagnostic tests.
- These problems are modelled as binary classification tasks, although may require specialized techniques.

Classification Models

- Decision Tree Classification
- K-Nearest Neighbors
- Logistic Regression
- Naïve Bayes
- Random Forest Classification
- Support Vector Machines

Applications of Classification Algorithms

Some of the applications of Classification models are:

- Sentiment Analysis
- Email Spam Classification
- Document Classification
- Image Classification

Sentiment Analysis

Sentiment analysis is a machine learning text analysis technique that assigns sentiment (opinion, feeling, or emotion) to words within a text, or an entire text, on a polarity scale of *Positive*, *Negative*, or *Neutral*.

It can automatically read through thousands of pages in minutes or constantly monitor social media for posts about you.

Using advanced machine learning algorithms, sentiment analysis models can be trained to read for things like sarcasm and misused or misspelled words. Once properly trained, models produce consistently accurate results in a fraction of the time.

Email Spam Classification

One of the most common uses of classification, working non-stop and with little need for human interaction, email spam classification saves us from tedious deletion tasks and sometimes even costly phishing scams.

Email applications use the above algorithms to calculate the likelihood that an email is either not intended for the recipient or unwanted spam. Using text analysis classification techniques, spam emails are weeded out from the regular inbox: perhaps a recipient's name is spelled incorrectly, or certain scamming keywords are used.

Document Classification

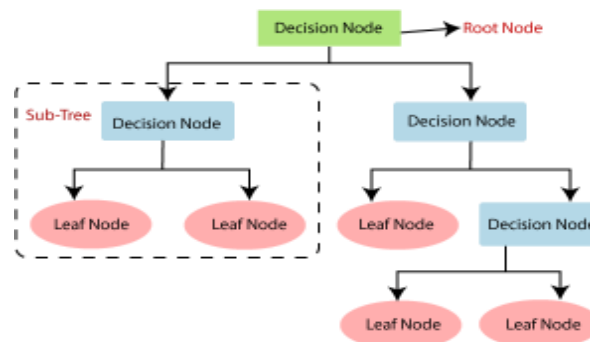
Document classification is the ordering of documents into categories according to their content. This was previously done manually, as in the library sciences or hand-ordered legal files. Machine learning classification algorithms, however, allow this to be performed automatically. Document classification differs from text classification, in that, entire documents, rather than just words or phrases, are classified. This is put into practice when using search engines online, cross-referencing topics in legal documents, and searching healthcare records by drug and diagnosis.

Image Classification

Image classification assigns previously trained categories to a given image. These could be the subject of the image, a numerical value, a theme, etc. Image classification can even use multi-label image classifiers, that work similarly to multi-label text classifiers, to tag an image of a stream, for example, into different labels, like “stream,” “water,” “outdoors,” etc.

Decision Trees

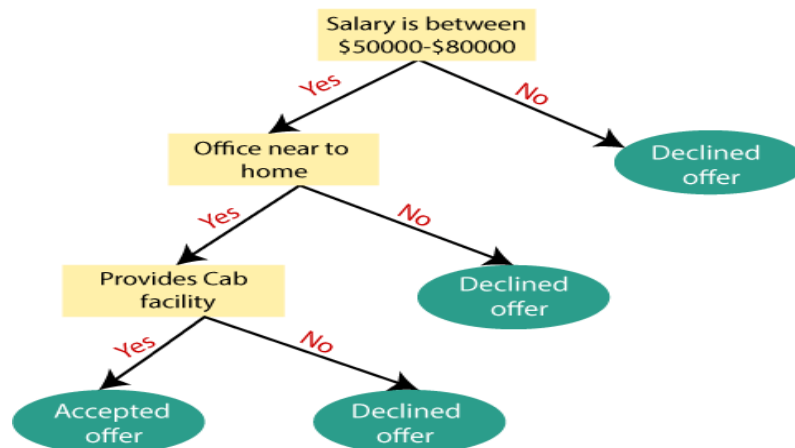
- ❖ Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.
- ❖ It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.
- ❖ In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- ❖ It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.
- ❖ It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.



Decision Tree Terminologies

- **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree:** A tree formed by splitting the tree.
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.
- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer).

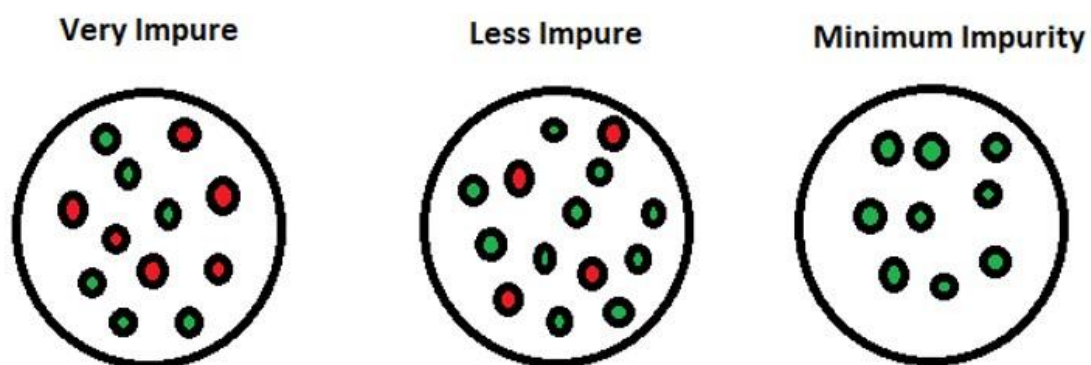


Understanding Entropy, information gain

What is Entropy and Information Gain? How are they used to construct decision trees?

What is Entropy?

Entropy is the measure of impurity, disorder or uncertainty in a bunch of examples. Entropy controls how a Decision Tree decides to split the data. The below image shows impurity level of each set.

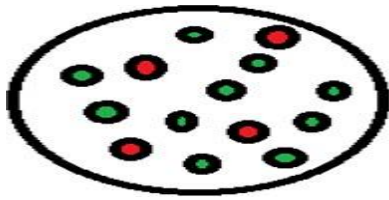


If we have a set of K different values, then we can calculate the entropy using this formula:

$$\text{entropy} = - \sum_{i=1}^k P(\text{value}_i) \cdot \log_2(P(\text{value}_i))$$

where, $P(\text{value}_i)$ is the probability of getting the i th value when randomly selecting one from the set.

For eg, let's take the following image with green and red circles.



In this group, we have 14 circles, out of which 10 are green (10/14) and 4 are red (4/14). Let's find the entropy of this group.

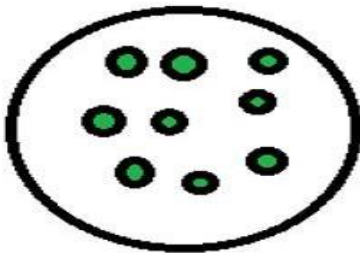
$$\text{Entropy}(\text{Green}) = \log_2(10/14) = -0.4854268272$$

$$\text{Entropy}(\text{Red}) = \log_2(4/14) = -1.807354922$$

$$\text{Entropy}(\text{Set}) = -(10/14)(-0.4854268272) - (4/14)(-1.807354922) = 0.86$$

The entropy of a group in which all examples belong to the same class will always be 0 as shown below:

Minimum Impurity

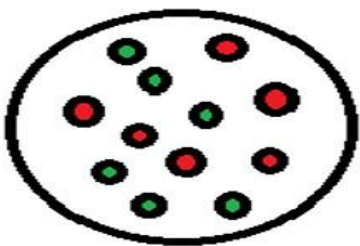


$$\text{Entropy} = -1 \log_2 1 = 0$$

This is not a good dataset for learning.

The entropy of a group with 50% in either class will always be 1 as shown below:

Very Impure



$$\text{Entropy} = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

Such a set is good for learning.

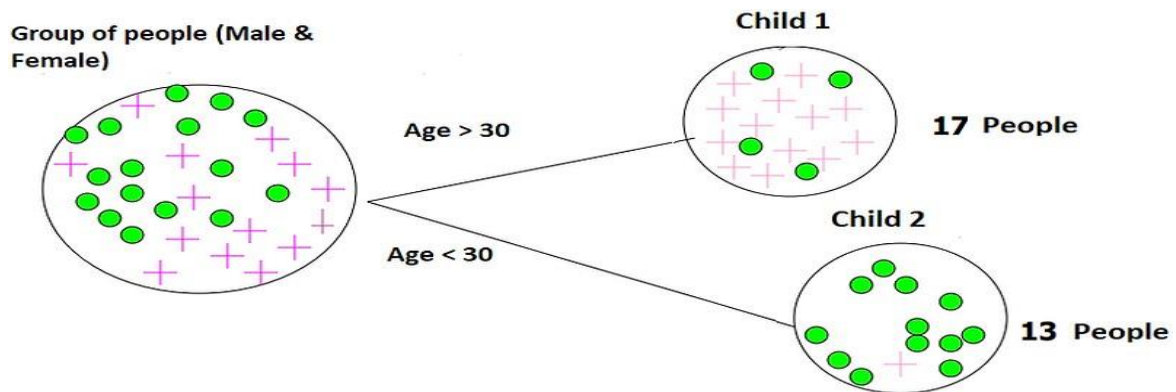
What is Information Gain?

Information gain (IG) measures how much “information” a feature gives us about the class. It tells us how important a given attribute of the feature vectors is. **Information gain (IG)** is used to decide the ordering of attributes in the nodes of a decision tree. Information gain (IG) is calculated as follows:

$$\text{Information Gain} = \text{entropy}(\text{parent}) - [\text{average entropy}(\text{children})]$$

Let's look at an example to demonstrate how to calculate Information Gain.

Let's say a set of 30 people both Male and female are split according to their age. Each person's age is compared to 30 and they are separated into 2 child groups as shown in the image and their corresponding node's entropy is calculated. The main node is called the Parent node and the 2 sub nodes are called child nodes.



The entropies of parent and child nodes are calculated as shown below. The Information gain is then calculated using the entropy of individual nodes.

$$\text{parent entropy} = -\left(\frac{14}{30} \cdot \log_2 \frac{14}{30}\right) - \left(\frac{16}{30} \cdot \log_2 \frac{16}{30}\right) = \mathbf{0.996}$$

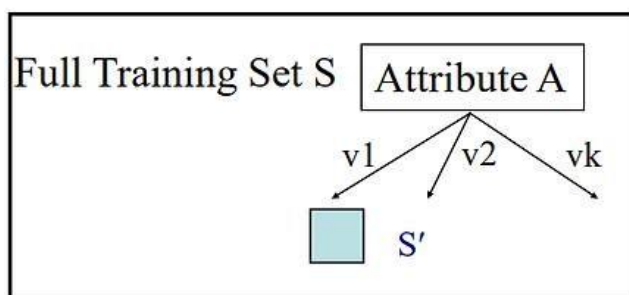
$$\text{child 1 entropy} = -\left(\frac{13}{17} \cdot \log_2 \frac{13}{17}\right) - \left(\frac{4}{17} \cdot \log_2 \frac{4}{17}\right) = \mathbf{0.787}$$

$$\text{child 2 entropy} = -\left(\frac{12}{13} \cdot \log_2 \frac{12}{13}\right) - \left(\frac{1}{13} \cdot \log_2 \frac{1}{13}\right) = \mathbf{0.391}$$

$$\text{(Weighted) Average Entropy of children} = \left(\frac{17}{30} \cdot 0.787\right) + \left(\frac{13}{30} \cdot 0.391\right) = 0.615$$

$$\text{Information Gain} = 0.996 - 0.615 = \mathbf{0.38} \text{ for this split}$$

The steps that needs to be followed to construct a decision tree using Information gain is shown below:



Step 1:

Choose the attribute with highest Information gain from the set as root node.

Step 2:

Construct child nodes for each value of A

Step 3:

Repeat recursively until the whole tree is built

Entropy and Information Gain are two main concepts that are used when it comes to constructing a decision tree, to determine the nodes and the best way to split.

How to Stop Overfitting

Identify overfitting

Overfitting in machine learning occurs when a model fits the training data too well, and as a result can't accurately predict on unseen test data. In other words, the model has simply memorized specific patterns and noise in the training data but is not flexible enough to make predictions on real data.

Consider the following trained models and their corresponding train and test accuracies.

Model	Train accuracy	Test accuracy
A	99.9%	95%
B	87%	87%
C	99.9%	45%

Considering model **A**, there is a common misconception that if test accuracy on unseen data is lower than training accuracy, the model is overfitted. However, test accuracy should always be less than training accuracy.

When comparing models **A** and **B**, model **A** is a better model because it has higher test accuracy, and although the test accuracy is slightly lower at 95%, it is not a significant difference that suggests overfitting is present. You wouldn't choose model **B** simply because the train and test accuracies are closer together.

Model **C** represents a clear case of overfitting; the training accuracy is very high but the test accuracy isn't anywhere near as high. This distinction is subjective, but comes from knowledge of your problem and data, and what magnitudes of error are acceptable.

Prevent Overfitting

The best way to prevent overfitting is to follow ML best-practices including:

- Using more training data
- Early Stopping
- Preventing target leakage
- Using fewer features
- Cross-validation**

1. Use more data/ Add training data

Obviously, the best solution would be to increase the size of the training data. Having more samples on the training set, allows the model to be more efficient.

Using **more data** is the simplest and best possible way to prevent overfitting, and as an added bonus typically increases accuracy. When you use more data, it becomes harder for the model to memorize exact patterns, and it is forced to reach solutions that are more flexible to accommodate more conditions.

Early Stopping

In this technique, the training is paused before the model starts learning the noise within the model. In this process, while training the model iteratively, measure the performance of the model after each iteration. Continue up to a certain number of iterations until a new iteration improves the performance of the model.

After that point, the model begins to overfit the training data; hence we need to stop the process before the learner passes that point.

Prevent target leakage

Target leakage is a similar issue, where you may not see overfitting between train/test sets, but rather it appears at prediction-time. Target leakage occurs when your model "cheats" during training by having access to data that it shouldn't normally have at prediction -time. For example, if your problem is to predict on Monday what a commodity price will be on Friday, but one of your features accidentally included data from Thursdays, that would be data the model won't have at prediction-time since it cannot see into the future. Target leakage is an easy mistake to miss but is often characterized by abnormally high accuracy for your problem.

Use fewer features

Removing features can also help with overfitting by preventing the model from having too many fields to use to memorize specific patterns, thus causing it to be more flexible. It can be difficult to measure quantitatively, but if you can remove features and retain the same accuracy, you have likely made the model more flexible and have reduced the risk of overfitting.

Cross-validation

Cross-validation (CV) is the process of taking many subsets of your full training data and training a model on each subset. The idea is that a model could get "lucky" and have great accuracy with one subset, but by using many subsets the model won't achieve this high accuracy every time. When doing CV, you provide a validation holdout dataset, specify your CV folds (number of subsets) and automated ML will train your model and tune hyperparameters to minimize error on your validation set. One CV fold could be overfitted, but by using many of them it reduces the probability that your final model is overfitted. The tradeoff is that CV does result in longer training times and thus greater cost, because instead of training a model once, you train it once for each n CV subsets.

Pruning

Pruning is a data compression technique in machine learning and search algorithms that reduces the size of decision trees by removing sections of the tree that are non-critical and redundant to classify instances.

1. Post Pruning :

- This technique is used after construction of decision tree.
- This technique is used when decision tree will have very large depth and will show overfitting of model.
- It is also known as backward pruning.
- This technique is used when we have infinitely grown decision tree.
- Here we will control the branches of decision tree that is `max_depth` and `min_samples_split` using **cost_complexity_pruning**

2. Pre-Pruning :

- This technique is used before construction of decision tree.
- Pre-Pruning can be done using **Hyperparameter tuning**.
- Overcome the overfitting issue.
- GridSearchCV for **Hyperparameter** tuning can be used.

Decision Tree Classifier

The Scikit-learn library provides the module name **DecisionTreeClassifier** for performing multiclass classification on dataset.

How Does it Works

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM) like Information Gain or Gini Index**.
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Parameters

Following table consist the parameters used by **sklearn.tree.DecisionTreeClassifier** module

Sr.No	Parameter & Description
1	<i>criterion</i> – string, optional default= “gini” It represents the function to measure the quality of a split. Supported criteria are “gini” and “entropy”. The default is gini which is for Gini impurity while entropy is for the information gain.
2	<i>max_depth</i> – int or None, optional default=None This parameter decides the maximum depth of the tree. The default value is None which means the nodes will expand until all leaves are pure or until all leaves contain less than min_smamples_split samples.
3	<i>min_samples_split</i> – int, float, optional default=2 This parameter provides the minimum number of samples required to split an internal node.
4	<i>min_samples_leaf</i> – int, float, optional default=1 This parameter provides the minimum number of samples required to be at a leaf node.
5	<i>max_features</i> – int, float, string or None, optional default=None It gives the model the number of features to be considered when looking for the best split.
6	<i>max_leaf_nodes</i> – int or None, optional default=None This parameter will let grow a tree with max_leaf_nodes in best-first fashion. The default is none which means there would be unlimited number of leaf nodes.

Evaluation metrics for classification

Accuracy

- It is most common performance metric for classification algorithms. It may be defined as the number of correct predictions made as a ratio of all predictions made.
- Accuracy is useful when the target class is **well balanced** but is not a good choice for the unbalanced classes.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Accuracy is useful when the target class is **well balanced** but is not a good choice for the unbalanced classes.

Confusion Matrix

- It is the easiest way to measure the performance of a classification problem where the output can be of two or more type of classes.
- A confusion matrix is nothing but a table with two dimensions viz. “Actual” and “Predicted” and furthermore, both the dimensions have “True Positives (TP)”, “True Negatives (TN)”, “False Positives (FP)”, “False Negatives (FN)” as shown below .

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Explanation of the terms associated with confusion matrix are as follows –

- True Positives (TP)** – It is the case when both actual class & predicted class of data point is 1.
- True Negatives (TN)** – It is the case when both actual class & predicted class of data point is 0.
- False Positives (FP)** – It is the case when actual class of data point is 0 & predicted class of data point is 1.

- False Negatives (FN) – It is the case when actual class of data point is 1 & predicted class of data point is 0

Precision

- Precision, used in document retrievals, may be defined as the number of correct documents returned by our ML model.
- Precision is useful in the cases where False Positive is a higher concern than False Negatives.
- Precision explains how many of the correctly predicted cases actually turned out to be positive.
- The importance of Precision is in music or video recommendation systems, e-commerce websites, etc. where wrong results could lead to customer churn and this could be harmful to the business.

$$\text{Precision} = \frac{\text{TruePositive}}{\text{TruePositive} + \text{FalsePositive}}$$

Recall or Sensitivity

- Recall explains how many of the actual positive cases we were able to predict correctly with our model. It is a useful metric in cases where False Negative is of higher concern than False Positive.
- It is important in medical cases where it doesn't matter whether we raise a false alarm but the actual positive cases should not go undetected!*

$$\text{Recall} = \frac{TP}{TP + FN}$$

Specificity

- Specificity, in contrast to recall, may be defined as the number of negatives returned by our ML model. We can easily calculate it by confusion matrix with the help of following formula

$$\text{Specificity} = \frac{TN}{TN + FP}$$

F1-Score

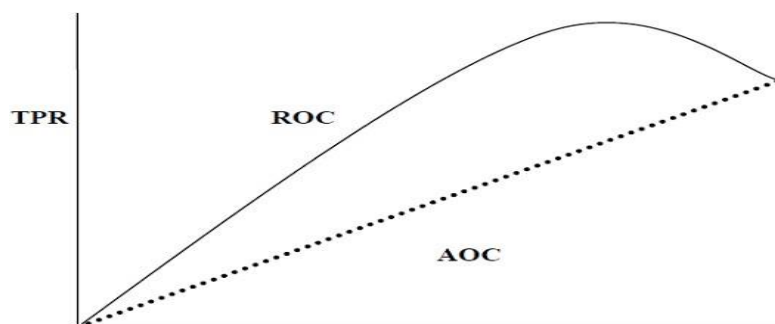
- ▮ This score will give us the harmonic mean of precision and recall. Mathematically, F1 score is the weighted average of the precision and recall. The best value of F1 would be 1 and worst would be 0. We can calculate F1 score with the help of following formula

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall}$$

- ▮ F1 score is having equal relative contribution of precision and recall.
- ▮ We can use `classification_report` function of `sklearn.metrics` to get the classification report of our classification model.

AUC (Area Under ROC curve)

- ▮ AUC (Area Under Curve)-ROC (Receiver Operating Characteristic) is a performance metric, based on varying threshold values, for classification problems. As name suggests, ROC is a probability curve and AUC measure the separability.
- ▮ In simple words, AUC-ROC metric will tell us about the capability of model in distinguishing the classes. Higher the AUC, better the model.
- ▮ Mathematically, it can be created by plotting TPR (True Positive Rate) i.e. Sensitivity or recall vs FPR (False Positive Rate) i.e. 1-Specificity, at various threshold values.



Build Decision Tree Model to Predict House Price on houseprice.csv dataset

1. FIRST IMPORT THE LIBRARIES AND LOAD THE DATASET AND ARRANGE DATA INTO FEATURES AND TARGET

Scikit-learn's `train_test_split` expects data in the form of features and target. In scikit-learn, a features matrix is a two-dimensional grid of data where rows represent samples and columns represent features. A target is what you want to predict from the data. In the below example we use price as a target.

```
features = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors']
```

```
X = df.loc[:, features]
```

```
y = df.loc[:, ['price']]
```

`train_test_split` splits the data and returns a list which contains four NumPy [arrays](#), while `train_size = .75` puts 75 percent of the data into a training set and the remaining 25 percent into a testing set.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, train_size = .75)
```

2. IMPORT THE MODEL YOU WANT TO USE

In scikit-learn, all machine learning models are implemented as Python classes.

```
from sklearn.tree import DecisionTreeRegressor
```

3. MAKE AN INSTANCE OF THE MODEL

In the code below, I set the hyperparameter `max_depth = 2` to pre-prune my tree to make sure it doesn't have a depth greater than two.

The `random_state` is a pseudo-random number parameter that allows you to reproduce the same train test split each time you run the code.

```
reg = DecisionTreeRegressor(max_depth = 2, random_state = 0)
```

4. TRAIN THE MODEL ON THE DATA

Train the model on the data, storing the information learned from the data.

```
reg.fit(X_train, y_train)
```

5. PREDICT LABELS OF UNSEEN TEST DATA

```
reg.predict(x_test[0:10])
```

```
array([ 406622.58288211, 1095030.54807692, 406622.58288211,
        406622.58288211, 657115.94280443, 406622.58288211,
        406622.58288211, 657115.94280443, 657115.94280443,
        1095030.54807692])
```

Lets consider a single observation/house and then proceed to examine how the model makes its prediction.

```
X_test.head(1)
```

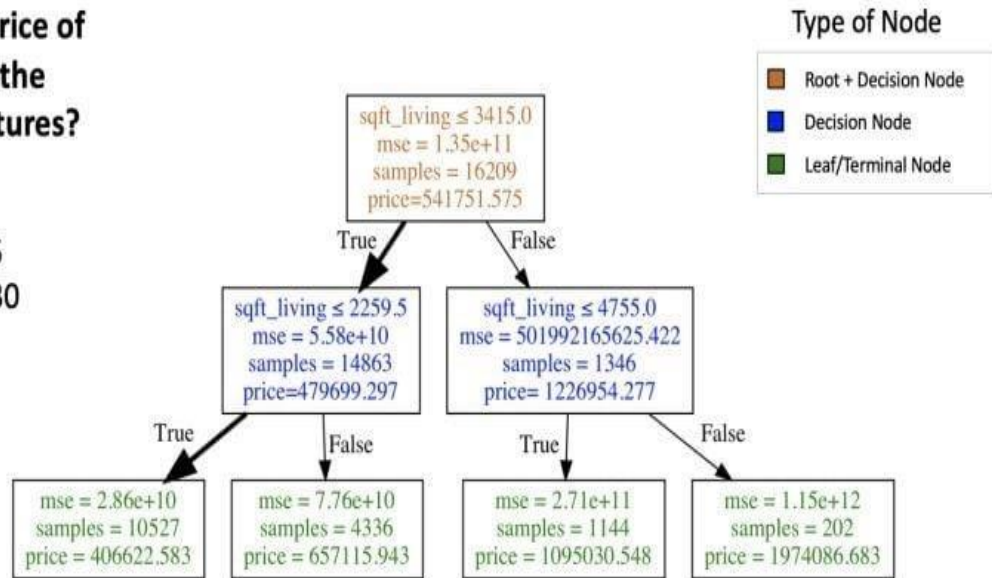
	bedrooms	bathrooms	sqft_living	sqft_lot	floors
17384	2	1.5	1430	1650	3.0

```
reg.predict(X_test.iloc[0].values.reshape(1, -1))
```

```
array([ 406622.58288211])
```

What is the price of a house with the following features?

bedrooms: 2
bathrooms: 1.5
sqft living: 1430
sqft lot: 1650
floors: 3



The price prediction is 406622.583 which was based on the average price of 10527 houses (samples) contained in the leaf node.

Complete Code To Build House Price Prediction Model

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
url='https://raw.githubusercontent.com/mGalarnyk/Tutorial_Data/master/King_County/kingCountyHouseData.csv'
df = pd.read_csv(url)
# Selecting columns I am interested in
columns = ['bedrooms','bathrooms','sqft_living','sqft_lot','floors','price']
df = df.loc[:, columns]
df.head(10)
features = ['bedrooms','bathrooms','sqft_living','sqft_lot','floors']
X = df.loc[:, features]
y = df.loc[:, ['price']]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, train_size = .75)
from sklearn.tree import DecisionTreeRegressor
reg = DecisionTreeRegressor(max_depth = 2, random_state = 0)
reg.fit(X_train, y_train)
reg.predict(X_test[0:10])
  
```

```
X_test.head(1)
reg.predict(X_test.iloc[0].values.reshape(1,-1))
score = reg.score(X_test, y_test)
print(score*100)
```

Build decision tree-based model in python for Breast Cancer Wisconsin (diagnostic) dataset from sci-kit learn

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import warnings
warnings.filterwarnings("ignore")

# loading dataset
df = pd.read_csv("C:/Users/Hp/OneDrive/Desktop/Datasets/data.csv")
display(df)
df = df.drop(["Unnamed: 32"], axis=1)
df.info()
diagnosis = df['diagnosis']
features = df.drop('diagnosis',axis=1)

# splitting the data into training and testing sets
features_train,features_test,labels_train,labels_test=train_test_split(features,diagnosis,test_size = 0.1)

# creating the classifier
clf = tree.DecisionTreeClassifier(min_samples_split = 10)
clf.fit(features_train,labels_train)

pred = clf.predict(features_test)

print("Accuracy =", accuracy_score(pred, labels_test)*100, "%")
```

Build and Evaluate Decision Tree Model on iris dataset

```
import pandas as pd
from sklearn import tree
df = pd.read_csv('C:/Users/Hp/OneDrive/Desktop/Datasets/iris.csv')
df.info()

# Decision tree
```

```

from sklearn.model_selection import train_test_split
decision = tree.DecisionTreeClassifier(criterion='gini')
features = ['SepalLengthCm','SepalWidthCm','PetalLengthCm','PetalWidthCm']
X = df.loc[:, features]
Y = df.loc[:, ['Species']]
trainX, testX, trainY, testY = train_test_split(X, Y, test_size=0.25)
decision.fit(trainX, trainY)
y_pred = decision.predict(testX)
y_score = decision.score(testX, testY)
print('Accuracy: ', y_score)
# Compute the average precision score and recall score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
micro_precision = precision_score(y_pred, testY, average='micro')
print('Micro-averaged precision score:',micro_precision)

macro_precision = precision_score(y_pred, testY, average='macro')
print('Macro-averaged precision score:',macro_precision)

per_class_precision = precision_score(y_pred, testY, average=None)
print('Per-class precision score:', per_class_precision)
micro_recall = recall_score(y_pred, testY, average='micro')
print('Micro-averaged recall score:',micro_recall)
macro_recall = recall_score(y_pred, testY, average='macro')
print('Macro-averaged recall score:',macro_recall)
per_class_recall = recall_score(y_pred, testY, average=None)
print('Per-class recall score:', per_class_recall)

```

OUTPUT

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id              150 non-null   int64
1   SepalLengthCm   150 non-null   float64
2   SepalWidthCm    150 non-null   float64
3   PetalLengthCm   150 non-null   float64
4   PetalWidthCm    150 non-null   float64
5   Species         150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
Accuracy:  0.9210526315789473
Micro-averaged precision score: 0.9210526315789473
Macro-averaged precision score: 0.9285714285714285
Per-class precision score: [1.          1.          0.78571429]
Micro-averaged recall score: 0.9210526315789473
Macro-averaged recall score: 0.9375
Per-class recall score: [1.          0.8125  1.          ]

```

Program to demonstrate evaluation metrics using placement dataset.

```
# import modules
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
dataset = pd.read_csv("C:/Users/Shilpa/Desktop/placement_dataset.csv")
print(dataset)
del dataset['sl_no']
# catgorising col for further labelling
dataset["gender"] = dataset["gender"].astype('category')
dataset["ssc_b"] = dataset["ssc_b"].astype('category')
dataset["hsc_b"] = dataset["hsc_b"].astype('category')
dataset["degree_t"] = dataset["degree_t"].astype('category')
dataset["workex"] = dataset["workex"].astype('category')
dataset["specialisation"] = dataset["specialisation"].astype('category')
dataset["status"] = dataset["status"].astype('category')
dataset["hsc_s"] = dataset["hsc_s"].astype('category')
dataset.dtypes
# labelling the columns
dataset["gender"] = dataset["gender"].cat.codes
dataset["ssc_b"] = dataset["ssc_b"].cat.codes
dataset["hsc_b"] = dataset["hsc_b"].cat.codes
dataset["degree_t"] = dataset["degree_t"].cat.codes
dataset["workex"] = dataset["workex"].cat.codes
dataset["specialisation"] = dataset["specialisation"].cat.codes
dataset["status"] = dataset["status"].cat.codes
dataset["hsc_s"] = dataset["hsc_s"].cat.codes
# display dataset
Print(Dataset)
# selecting the features and labels
X = dataset.iloc[:, :-1].values
Y = dataset.iloc[:, -1].values
# display dependent variables
Print(Y)
# dividing the data into train and test
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2)
# display dataset
dataset.head()
# creating a classifier using sklearn
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier().fit(X_train, Y_train)
# printing the acc
```

```
clf.score(X_test, Y_test)
# predicting for random value
clf.predict([[0, 87, 0, 95, 0, 2, 78, 2, 0, 0, 1, 0]])
# creating a Y_pred for test data
Y_pred = clf.predict(X_test)
# display predicted values
Print(Y_pred)
# evaluation of the classifier
from sklearn.metrics import confusion_matrix, accuracy_score
# display confusion matrix print(confusion_matrix(Y_test, Y_pred))
# display accuracy
print(accuracy_score(Y_test, Y_pred))
```

Hyperparameter tuning in decision tree classifier

What is tuning?

Tuning is **the process of maximizing a model's performance without overfitting or creating too high of a variance**. In machine learning, this is accomplished by selecting appropriate "hyperparameters." Hyperparameters can be thought of as the "dials" or "knobs" of a machine learning model.

What are hyperparameters?

"Hyperparameters are defined as the parameters that are explicitly defined by the user to control the learning process."

Here the prefix "hyper" suggests that the parameters are top-level parameters that are used in controlling the learning process. The value of the Hyperparameter is selected and set by the machine learning engineer before the learning algorithm begins training the model. **Hence, these are external to the model, and their values cannot be changed during the training process.**

Examples of hyper parameters in decision tree model:- criterion, max_depth, min_sample_split, etc.

What is hyperparameter tuning?

Hyperparameter tuning is the process of **finding the configuration of hyperparameters that results in the best performance**.

Hyperparameter tuning is **choosing a set of optimal hyperparameters for a learning algorithm**. A hyperparameter is a model argument whose value is set before the learning process begins. The key to machine learning algorithms is hyperparameter tuning.

What is the importance of hyperparameter tuning?

Hyperparameters are **crucial as they control the overall behaviour of a machine learning model**. The ultimate goal is to find an optimal combination of hyperparameters that minimizes a predefined loss function to give better results.

What is the best strategy for hyperparameter tuning in machine learning?

Two best strategies for Hyperparameter tuning are:

GridSearchCV

RandomizedSearchCV.

What is Grid Search?

In the Grid Search, all the mixtures of hyperparameters combinations will pass through one by one into the model and check the score on each model. It gives us the set of hyperparameters which gives the best score. Scikit-learn package as a means of automatically iterating over these hyperparameters using cross-validation. This method is called Grid Search.

How does it work?

Grid Search takes the model or objects you'd prefer to train and different values of the hyperparameters. It then calculates the error for various hyperparameter values, permitting you to choose the best values.



Let the tiny circles represent different hyperparameters. We begin with one value for hyperparameters and train the model. We use different hyperparameters to train the model. We tend to continue the method until we have exhausted the various parameter values. Every model produces an error. We pick the hyperparameter that minimizes the error. To pick the hyperparameter, we split our dataset into 3 parts, the training set, validation set, and test set. We tend to train the model for different hyperparameters. We use the error component for each model. We select the hyperparameter that minimizes the error or maximizes the score on the validation set. In the end, we test our model performance using the test data.

Implement hyperparameter tuning using the sklearn library called gridsearchcv in Python.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
train_data=pd.read_csv("C:/Users/Hp/OneDrive/Desktop/Datasets/train.csv")
test_data=pd.read_csv("C:/Users/Hp/OneDrive/Desktop/Datasets/titanic_test.csv")
test_data.head()
```

	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

```
train_data.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
train_data=train_data.drop(['PassengerId','Name','SibSp','Parch','Ticket','Cabin','Embarked'],axis=1)
train_data.head()
```

	Survived	Pclass	Sex	Age	Fare
0	0	3	male	22.0	7.2500
1	1	1	female	38.0	71.2833
2	1	3	female	26.0	7.9250
3	1	1	female	35.0	53.1000
4	0	3	male	35.0	8.0500

```
test_data=test_data.drop(['Name','SibSp','Parch','Ticket','Cabin','Embarked'],axis=1)
test_data.head()
```

	Pclass	Sex	Age	Fare
0	3	male	34.5	7.8292
1	3	female	47.0	7.0000
2	2	male	62.0	9.6875
3	3	male	27.0	8.6625
4	3	female	22.0	12.2875

```
x_train=train_data.drop('Survived',axis=1)
x_train.replace({'male':0,'female':1},inplace=True)
test_data.replace({'male':0,'female':1},inplace=True)
y_train=train_data['Survived']
x_test=test_data.values
x_test
```

```
array([[ 3.      ,  0.      , 34.5     ,  7.8292],
       [ 3.      ,  1.      , 47.      ,  7.      ],
       [ 2.      ,  0.      , 62.      ,  9.6875],
       ...,
       [ 3.      ,  0.      , 38.5     ,  7.25    ],
       [ 3.      ,  0.      ,      nan,  8.05    ],
       [ 3.      ,  0.      ,      nan, 22.3583]])
```

```
test_data.isnull().sum()
```

```
Pclass      0
Sex          0
Age         86
Fare         1
dtype: int64
```

```
x_train.Age=x_train.Age.fillna(x_train.Age.mean())
test_data.Age=test_data.Age.fillna(test_data.Age.mean())
test_data.Fare=test_data.Fare.fillna(test_data.Fare.mean())
test_data.isnull().sum()
```

```
Pclass      0
Sex          0
Age          0
Fare         0
dtype: int64
```

```
dec=DecisionTreeClassifier()
dec.fit(x_train,y_train)
pred=dec.predict(test_data)
print(pred)
```

```
[0 0 1 1 1 0 0 0 1 0 0 0 1 1 1 1 0 1 1 0 0 1 1 0 1 0 1 1 1 0 0 0 1 0 1 0 0
 0 0 1 0 0 0 1 1 0 0 0 1 1 1 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 1 0 0 1 1 0 1 0
 1 0 0 1 0 1 1 0 0 0 0 0 1 1 0 1 1 0 1 0 0 1 1 0 1 0 1 0 0 0 1 0 0 0 1 0 0
 0 1 1 1 0 0 1 0 1 1 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0
 0 0 1 0 0 1 0 0 1 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 0 0 1 1 0 1 1 0 0 1 0 1
 0 1 0 0 0 0 0 1 0 1 0 1 0 0 0 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 1 0
 1 1 1 0 0 0 0 0 0 1 0 0 1 0 0 1 1 1 1 1 1 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 1
 0 0 0 1 1 0 0 0 0 0 0 0 0 0 1 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1 1 0 1 0 0 0 1 0 0
 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 1 1 0 0 0 0 0 1 0 0 1 0 1 1 1 1 0 0 0 1 1
 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 1 0 1 0 0 1 0 1 1 0 0 0
 0 0 1 0 1 0 0 1 0 0 0]
```

```
param_dict={"criterion":["gini","entropy"],
            "max_depth":range(1,10),
            "min_samples_split":range(1,10),
            "min_samples_leaf":range(1,5)}
grid=GridSearchCV(dec,param_grid=param_dict,cv=10,verbose=1,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
grid.best_params_
```

```
{'criterion': 'gini',
 'max_depth': 9,
 'min_samples_leaf': 3,
 'min_samples_split': 8}
```

```
grid.best_estimator_
```

```
DecisionTreeClassifier(max_depth=9, min_samples_leaf=3, min_samples_split=8)
```

```
grid.best_score_
```

```
0.8294756554307117
```

The confusion matrix for a Machine Learning Model is given below.
Evaluate the following's.

- (i) Accuracy
- (ii) Precision
- (ii) Recall
- (iv) Specificity
- (v) F1 - Score

		Actual	
		1	0
Predicted	1	45	8
	0	15	32

$$\begin{aligned}
 \text{(i) Accuracy} &= (TP + TN) / (TP + TN + FP + FN) \\
 &= (45+32) / (45 +32+8+15) \\
 &= 77/100 \\
 &= 0.77
 \end{aligned}$$

$$\begin{aligned}
 \text{(ii) Precision} &= TP / (TP + FP) \\
 &= 45/ (45+8) \\
 &= 45/53 \\
 &= 0.84
 \end{aligned}$$

$$\begin{aligned}
 \text{(ii) Recall} &= TP / (TP + FN) \\
 &= 45/ (45+15) \\
 &= 45/ 60 \\
 &= 0.75
 \end{aligned}$$

$$\begin{aligned}
 \text{(iv) Specificity} &= TN / (TN + FP) \\
 &= 32/ (32+8) \\
 &= 32/40 \\
 &= 0.80
 \end{aligned}$$

$$\begin{aligned}
 \text{(v) F1 – Score} &= 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall}) \\
 &= 2* (0.84 * 0.75) / (0.84+0.75) \\
 &= 2* (0.63) / (1.59) \\
 &= 2 * 0.396 \\
 &= 0.792
 \end{aligned}$$

A Machine learning model was built to classify patient as COVID +ve(1) or -ve(0). The confusion matrix for the model is as shown below. Evaluate accuracy, precision, recall, specificity and F1-Score.

		Actual	
		1	0
Predicted	1	397	103
	0	126	142

TruePositive(TP):397

True Negative(TN) : 142

FalsePositive(FP):103

FalseNegative(FN): 126

Accuracy: $\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$ $= \frac{397+142}{397+142+103+126}$ $= \frac{539}{768}$ $= 0.70$	Specificity: $\text{Specificity} = \frac{TN}{(TN+FP)}$ $= \frac{142}{(142+103)}$ $= \frac{142}{245}$ $= 0.57$ F1-Score : $\text{F1-Score} = 2 \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$ $= 2 \frac{(0.79 \times 0.75)}{(0.79 + 0.75)}$ $= 2 \frac{(0.5925)}{(1.54)}$ $= \frac{1.18}{1.54}$ $= 0.76$	Precision: $\text{Precision} = \frac{TP}{(TP+FP)}$ $= \frac{397}{(397+103)}$ $= \frac{397}{500}$ $= 0.79$ Recall: $\text{Recall} = \frac{TP}{(TP+FN)}$ $= \frac{397}{(397+126)}$ $= \frac{397}{523}$ $= 0.75$
--	--	--

7. (a) The confusion matrix looks like this

		1	0	
1	45	8		
0	15	32		

(i) Accuracy:

Accuracy measures the proportion of correctly classified samples out of the total number of samples.

Accuracy = $(TP + TN) / (TP + TN + FP + FN) = (45 + 32) / (45 + 8 + 15 + 32) = 77 / 100 \approx 0.77$ or 77%

(ii) Precision:

Precision is the proportion of true positive predictions out of the total positive predictions made by the model.

Precision = $TP / (TP + FP) = 45 / (45 + 8) \approx 0.8491$ or 84.91%

(iii) Recall (Sensitivity or True Positive Rate):

Recall measures the proportion of true positive predictions out of all the actual positive samples.

Recall = $TP / (TP + FN) = 45 / (45 + 15) \approx 0.75$ or 75%

(iv) Specificity (True Negative Rate):

Specificity measures the proportion of true negative predictions out of all the actual negative samples.

Specificity = $TN / (TN + FP) = 32 / (32 + 8) \approx 0.8$ or 80%

(v) F1-Score:

F1-score is the harmonic mean of precision and recall, providing a balanced metric for binary classification when the classes are imbalanced.

F1-Score = $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) = 2 * (0.8491 * 0.75) / (0.8491 + 0.75) \approx 0.796$ or 79.6%

5.a A Machine learning model was built to classify spam emails as “spam”(1) or “not spam”(0). The confusion matrix for the model is as shown below. Evaluate accuracy, precision, recall, specificity and F1-Score. 10 Marks

		Actual	
		1	0
Predicted	1	140	10
	0	5	50

Accuracy:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} = \frac{140+50}{140+50+10+5} = \frac{190}{205} \approx 0.9268$$

Precision:

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{140}{140+10} = \frac{140}{150} = 0.9333$$

Recall:

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{140}{140+5} = \frac{140}{145} \approx 0.9655$$

Specificity:

$$\text{Specificity} = \frac{TN}{TN+FP} = \frac{50}{50+10} = \frac{50}{60} \approx 0.8333$$

F1-Score:

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot 0.9333 \cdot 0.9655}{0.9333 + 0.9655} \approx 0.9491$$